

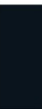


TCP: Ein Sliding-Window-Protokoll



Flusskontrolle in TCP

- TCP implementiert das Sliding-Window-Protokoll
 - Bei einer Fenstergröße von n können **n Bytes** verschickt werden, ohne dass ein ACK empfangen werden muss
 - Wenn der Empfang der Daten vom Empfänger bestätigt wurde, so können neue Daten versendet werden, es verschiebt sich das Fenster
- Nummerierung (zyklisch mit 32-Bit)
 - **Segmente** werden **durch** ihren **Byte-Offset** im Stream **identifiziert**, wobei die Startposition beim Verbindungsaufbau zufällig festgelegt wird
 - TCP verwendet **kumulative ACKs**: ACK $n+1$ sagt aus, dass **alle Daten von der vorigen logischen Position bis zur Position n korrekt** empfangen wurden und nun das Segment $n+1$ erwartet wird

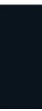




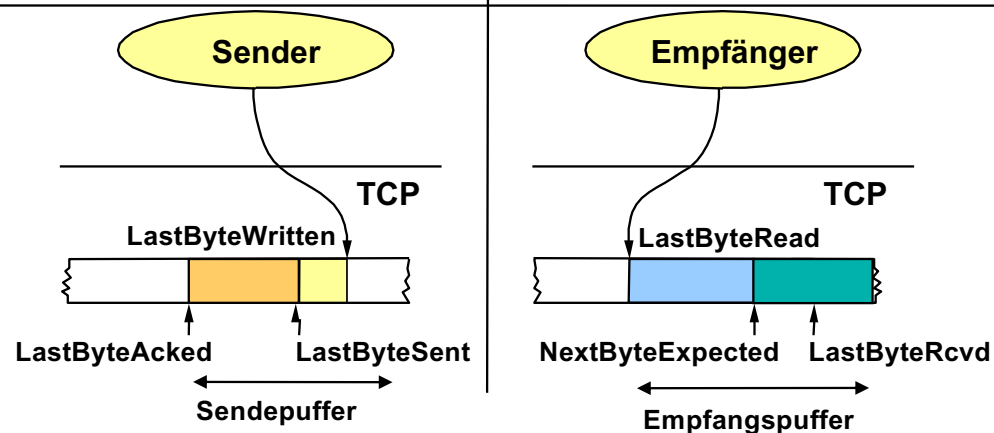
Besonderheiten

- Der Empfänger puffert außerhalb der Reihenfolge empfangene Segmente, so dass prinzipiell die Möglichkeit besteht, Selective Repeat/Reject durchzuführen
- Hinweis: TCP wird mangels echter NACK kein direktes Selective Reject nutzen, vielmehr werden mehrfach gleiche ACKs (die kumulativ sind) als NACK angesehen
- TCP verwendet eine **variable Fenstergröße**
 - Jede Bestätigung spezifiziert den **aktuell freien Platz** im Empfangspuffer (Advertised Window/Receiver Window)
 - Das Sliding Window des Senders wird somit durch die noch freie Pufferkapazität beim Empfänger beeinflusst

Wieso ist das wichtig?



Flusskontrolle in TCP: Aus dem Code



• Bedingungen für den Sender

$\text{LastByteAked} \leq \text{LastByteSent}$
 $\text{LastByteSent} \leq \text{LastByteWritten}$

Zwischenspeichern der Daten zwischen
 LastByteAked und LastByteWritten

$\text{LastByteSent} - \text{LastByteAked} < \text{AdvertisedWindow}$

$\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAked})$

■ Bedingungen für den Empfänger

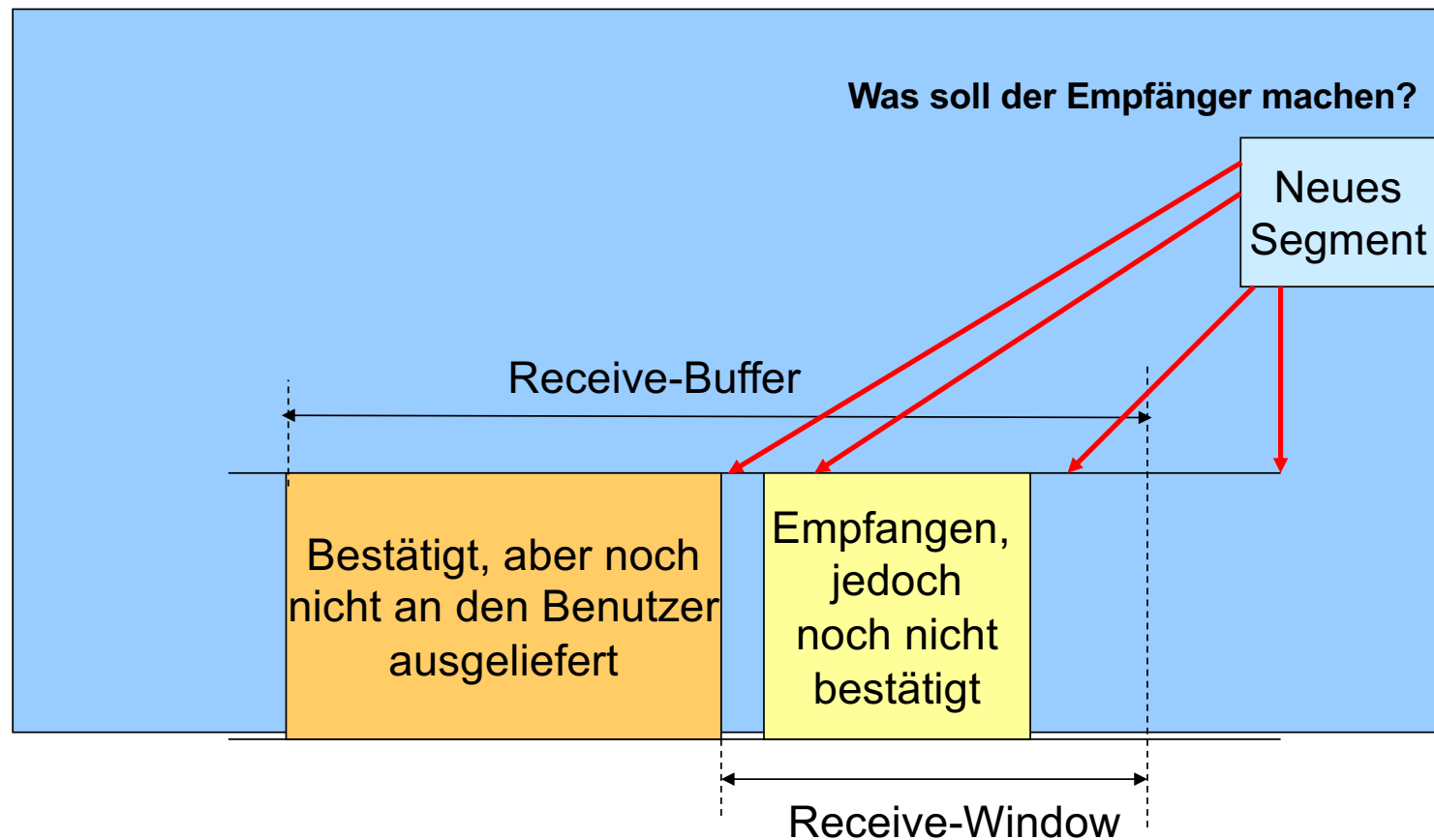
$\text{LastByteRead} < \text{NextByteExpected}$
 $\text{NextByteExpected} \leq \text{LastByteRcvd} + 1$

Zwischenspeichern der Daten zwischen
 LastByteRead and LastByteRcvd

$\text{AdvertisedWindow} = \text{Empfangspuffer} - ((\text{NextByteExpected} - 1) - \text{LastByteRead})$



Flusskontrolle: Empfänger-Seite





Transmission Control Protocol: Eine hybride Lösung

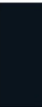
Go-Back-N/Selective Repeat Protokoll: TCP-Sender verwaltet lediglich einen Timer für das Segment, welches als nächstes bestätigt werden muss. Kommt es zu einem Time-Out, so wird augenscheinlich ein „Go-Back-N“ durchgeführt. **Tatsächlich** werden die bereits empfangenen Pakete zwischengespeichert, von daher kommt wird faktisch ein **Selective Repeat** durchgeführt

Selective Reject: Empfänger speichert nicht nur Out-of-Order Segmente im Empfangspuffer. Die meisten Versionen von TCP **emulieren** NAK-Mechanismen mittels **Triple-Duplicate-ACKs**:

Hierdurch kann ein **erneutes Übertragen eines Segments VOR Timeout** initiiert werden (beim 3. Duplicate Acknowledgement)

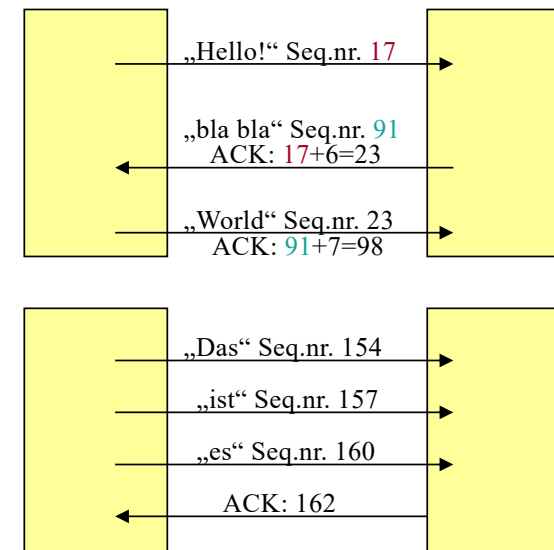
Wir haben somit **faktisch ein Selective Reject**

Merke: kumulative Bestätigungen ermöglichen das **Nutzen zwischengepufferter Daten** (Datenlücken können geschlossen werden)



Möglichst keine eigenständigen Bestätigungsnachrichten

- TCP unterstützt eine **bidirektionale** Kommunikation
- Eine Bestätigungsnachricht kann viele Segmente bestätigen
 - Kumulative Bestätigung
- Huckepack-Technik
 - Bestätigungen können auf dem Datenpaket der Gegenrichtung „reiten“
- Delayed Acknowledgments: Liegen keine Daten an, werden Acks verzögert – irgendwann aber doch verschickt
 - Versuch der Effizienzsteigerung
 - Ziel: Abwarten bis mit einer Bestätigung auch Daten versendet werden können
 - Bestätigungen nur für jedes zweite Segment





Reaktion des Empfängers (Delayed Acks)

Ereignis beim Empfänger	Aktion beim Empfänger
Ankunft eines Segments mit der erwarteten Sequenznummer. Alle Daten bis dahin sind jedoch schon bestätigt worden.	Delayed ACK. Warte bis zu 500ms auf das nächste Segment. Wenn dieses nicht empfangen wird, verschicke die Bestätigung.
Ankunft eines Segments mit der erwarteten Sequenznummer. Allerdings wurde noch keine Bestätigung des vorherigen Segments verschickt.	Sofortiges Verschicken einer kumulativen Bestätigung, die beide Segmente bestätigt.
Ankunft eines Segments hinter der erwarteten Segmentnummer. Es wird eine Lücke entdeckt.	Sofortiges Verschicken eines Duplicate ACK (DupACK). Dieses gibt die erwartete Segmentnummer an.
Ankunft eines Segments, das eine Lücke teilweise oder vollständig füllt.	Sofortiges Verschicken einer Bestätigung.





TCP Round Trip Time und Timeout

Q: Wann sollte es ein Timeout geben?

- RTT ist die Zeit, die es mindestens braucht, bis eine Bestätigung ankommen kann
- Ein Timeout sollte nicht vorher auftreten
- **Problem:** RTT variiert
 - Zu kurz: vorzeitiger Timeout und damit unnötige Neuübertragungen
 - Zu groß: sehr langsame Reaktion auf Paketverluste

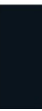
Q: Wie kann die RTT geschätzt werden?

- TCP stoppt die Zeit zwischen dem Versenden eines Segmentes und dem Empfang des ACKs
- Neuübertragungen gehen hier nicht ein, Delayed Acks müssen ausgeblendet werden
- **SampleRTT** wird zur Schätzung der RTT geglättet
- Hierdurch ergibt sich eine Mittelung über die Vergangenheit, und nicht nur das aktuelle **SampleRTT**



TCP ist eigentlich noch viel mehr...

- Der Verlust von Segmenten führt bei einem Timeout zum Go-Back-N (resp. Selective Repeat)
 - Man beginnt mit der Neuübertragung eines ganzen Windows
 - Haben sich Sender und Empfänger auf große Windows geeinigt, so werden erhebliche Mengen neu übertragen
 - Existieren viele solche Übertragungen im Netz, können Router überlastet werden
 - Als Resultat verwerfen sie Pakete, so dass auf TCP-Ebene ggf. keine Quittungen mehr eingehen und es wieder zu Timeouts kommt
- Reaktion auf Überlast durch Reduktion der Rate sinnvoll





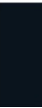
TCP: Flusskontrolle/Staukontrolle

Flusskontrolle:

- basiert auf dem Sliding-Window-Protokoll
- regelt den Datenfluss zwischen den Transportdienst-Nutzern
- Ack-getriebenes injizieren von Daten
- Übertragungsrate begrenzt auf ein Fenster pro RTT

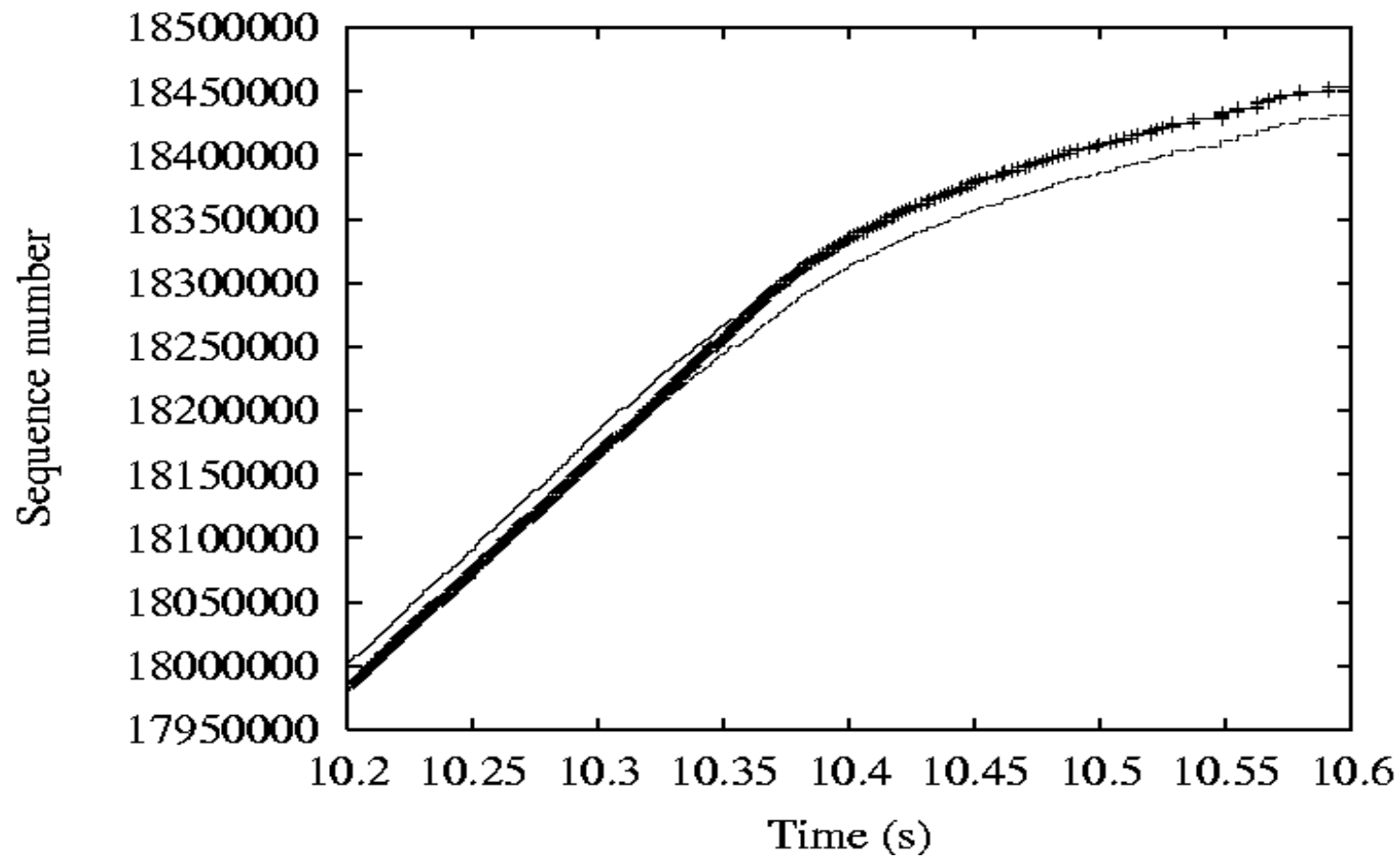
Staukontrolle:

- befasst sich mit Überlastsituationen in den Zwischensystemen (Routern)
- Überlastsituationen in Zwischensystemen führen letztlich zu Paketverlusten, so dass die entsprechenden Segmente nach einer gewissen Zeit erneut übertragen werden, bei Time-Out kommt es gar zu einem „Go-Back-N“/Selective Repeat
 - ⇒ Verstärkung der Überlastsituation durch permanente Neuübertragung (congestion collapse))!





Flusskontrolle in TCP: Self-Clocking

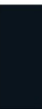




TCP: Staukontrolle

•Überlastfenster (Congestion-Window):

- Zusätzliche Beschränkung des verfügbaren Fensters auf Senderseite durch ein sogenanntes Congestion Window (es reicht eine cwin-Variable)
- Faktisch wird dieses zumeist **nicht in Bytes sondern in Anzahl der maximalen Segmentgröße** geführt (**Maximum Segment Size**)
- Hintergrund dieser Regel ist, dass die Verfahren zur Änderung der cwin-Variable die Anzahl der ausstehenden Segmente variieren
- Genau hier ist das Ziel, die Anzahl der erlaubten ausstehenden Segmente so zu variieren, dass Überlastsituationen vermieden werden
- Faktisch soll so auch Fairness zwischen TCP-Anwendungen erzielt werden





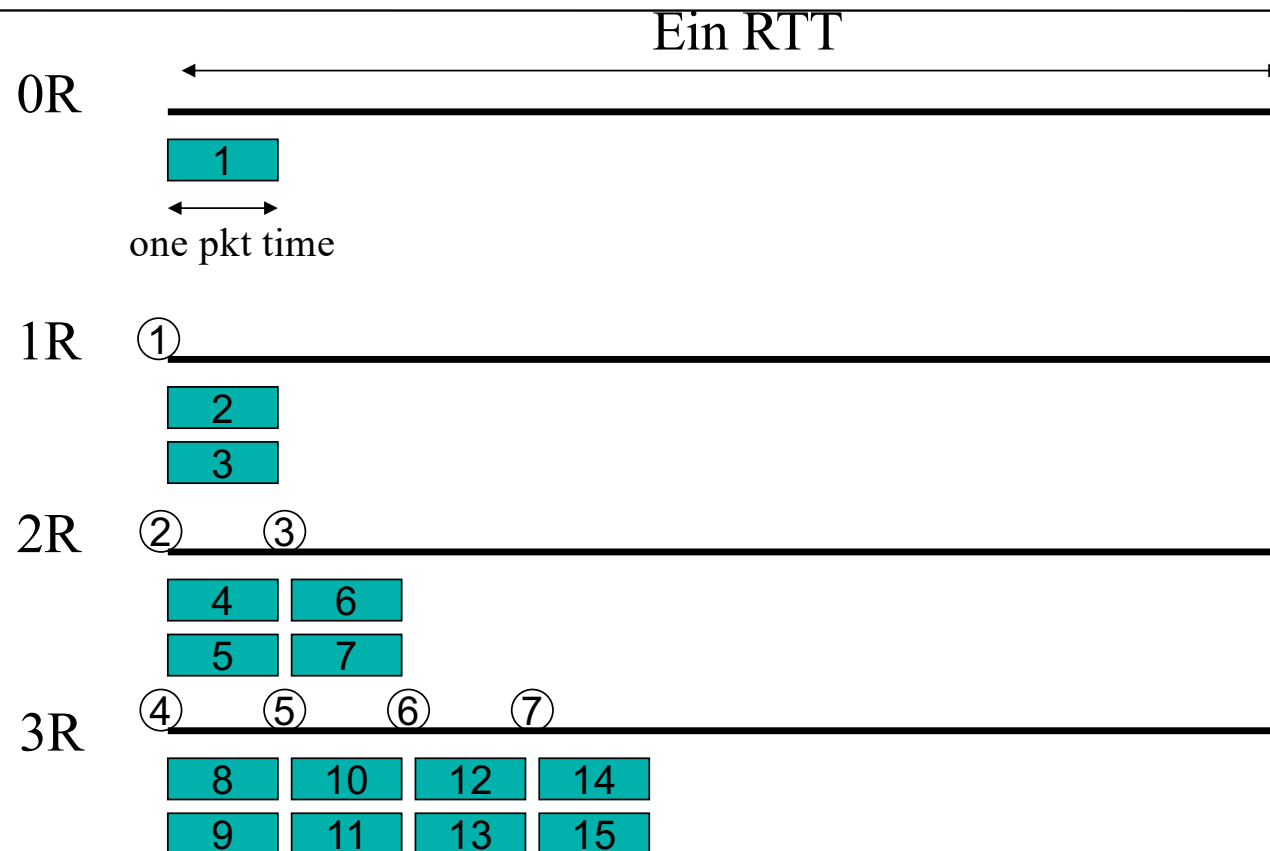
TCP: Staukontrolle

•Überlastfenster (Congestion-Window):

- Tatsächlich unterscheidet man zwei Strategien zur Änderung des Congestion-Window: die Slow-Start-Phase und die Congestion-Control-Phase
- Intern wird ein Threshold gepflegt, der das Vorgehen beeinflusst
$$ssthresh = \min (Receiver\ Window, Congestion\ Window * MSS) \text{ [Byte]}$$
- Congestion Window **bestimmt insbesondere zu Beginn die Übertragungsrate (Slow-Start)**
 - > Initiale Größe = 1 **Segment maximaler Größe**
 - > Größe des Überlastfensters wird in der Slow-Start-Phase bei jeder erfolgreichen Bestätigung um die Größe eines Segments erhöht
 - erfolgreiche Übertragung eines vollen Fensters von n Segmenten liefert $2n \Rightarrow \Rightarrow$ Verdopplung pro Runde $\Rightarrow$ exponentielles Wachstum
 - maximal bis Größe des Receiver-Window erreicht

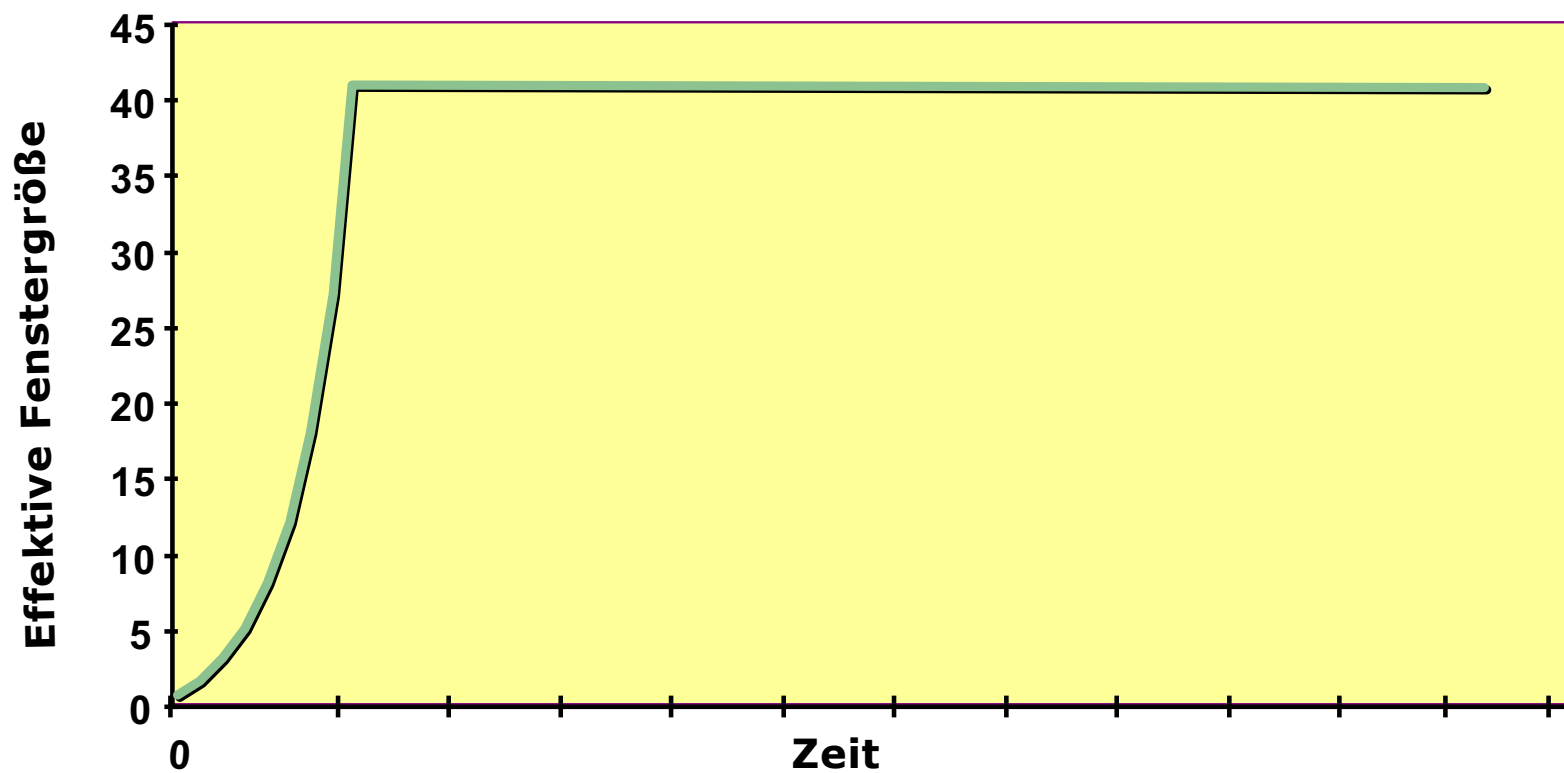


Slow-Start-Beispiel

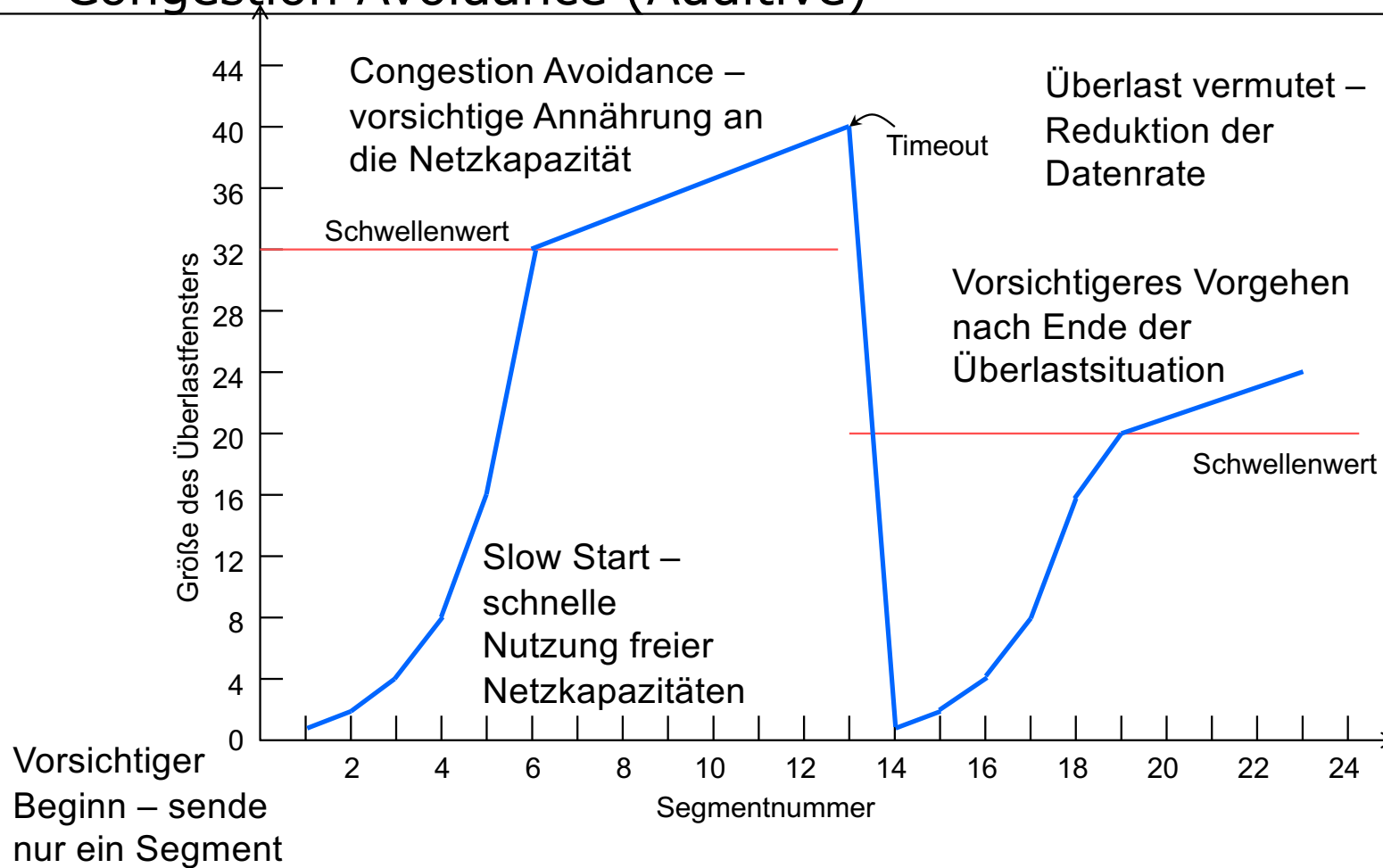




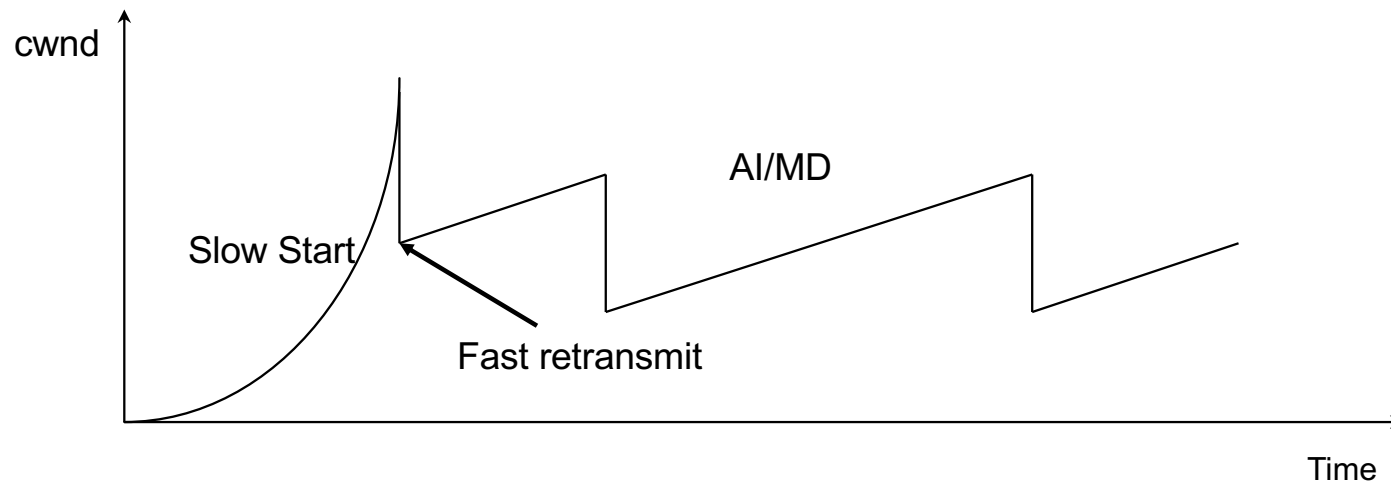
Staukontrolle – Slow-Start



TCP hat zwei Phasen: Slow Start (Multiplicative) Congestion Avoidance (Additive)



Fast Retransmit und Fast Recovery



Werden drei Duplicate Acks empfangen, so muss die Leitung noch etwas übertragen können.
Evtl. fehlt nur das eine Segment, welches dann übertragen wird
(Kein Go-Back-N, Selective Reject)
Hierbei verfolgt TCP eine Multiplicative Decrease Strategie (halbieren)



Staukontrolle bei TCP

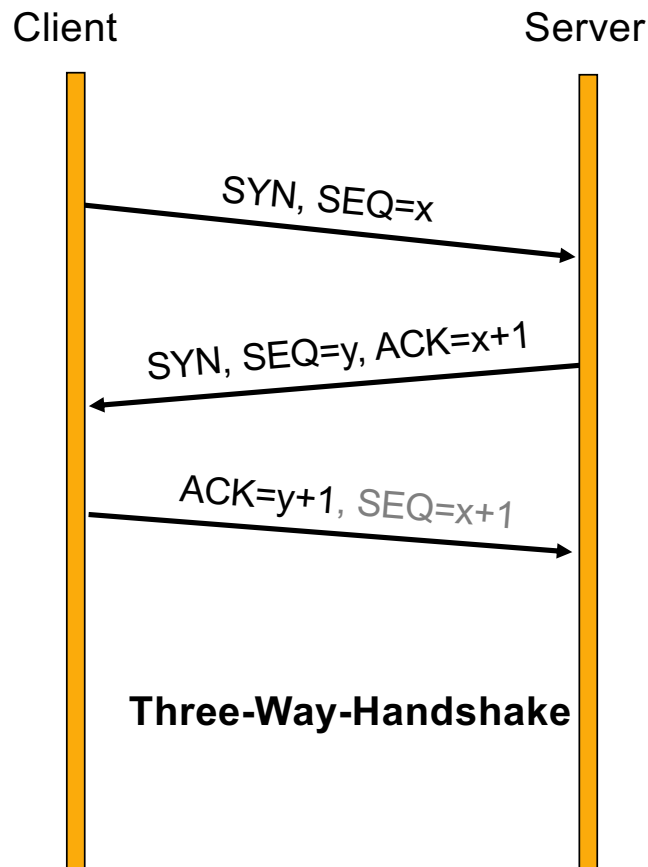
- Congestion Control ist deutlich komplexer als dargestellt
 - Viele Erweiterungen, um Einbruch der Datenrate zu vermeiden
 - Faktisch gibt es viele TCP-Varianten, die unterschiedliche Staukontrollverfahren implementieren

```
grep TCP_CONG /boot/config-$(uname -r)
CONFIG_TCP_CONG_ADVANCED=y
CONFIG_TCP_CONG_BIC=m
CONFIG_TCP_CONG_CUBIC=y
CONFIG_TCP_CONG_WESTWOOD=m
CONFIG_TCP_CONG_HTCP=m
CONFIG_TCP_CONG_HSTCP=m
CONFIG_TCP_CONG_HYBLA=m
CONFIG_TCP_CONG_VEGAS=m
CONFIG_TCP_CONG_SCALABLE=m
CONFIG_TCP_CONG_LP=m
CONFIG_TCP_CONG_VENO=m
CONFIG_TCP_CONG_YEAH=m
CONFIG_TCP_CONG_ILLINOIS=m
CONFIG_DEFAULT_TCP_CONG="cubic"
```

- Staukontrolle schafft Fairness! Wird eine Leitung von N-TCP-Verbindungen genutzt, so erhält jede $1/N$ -Anteil



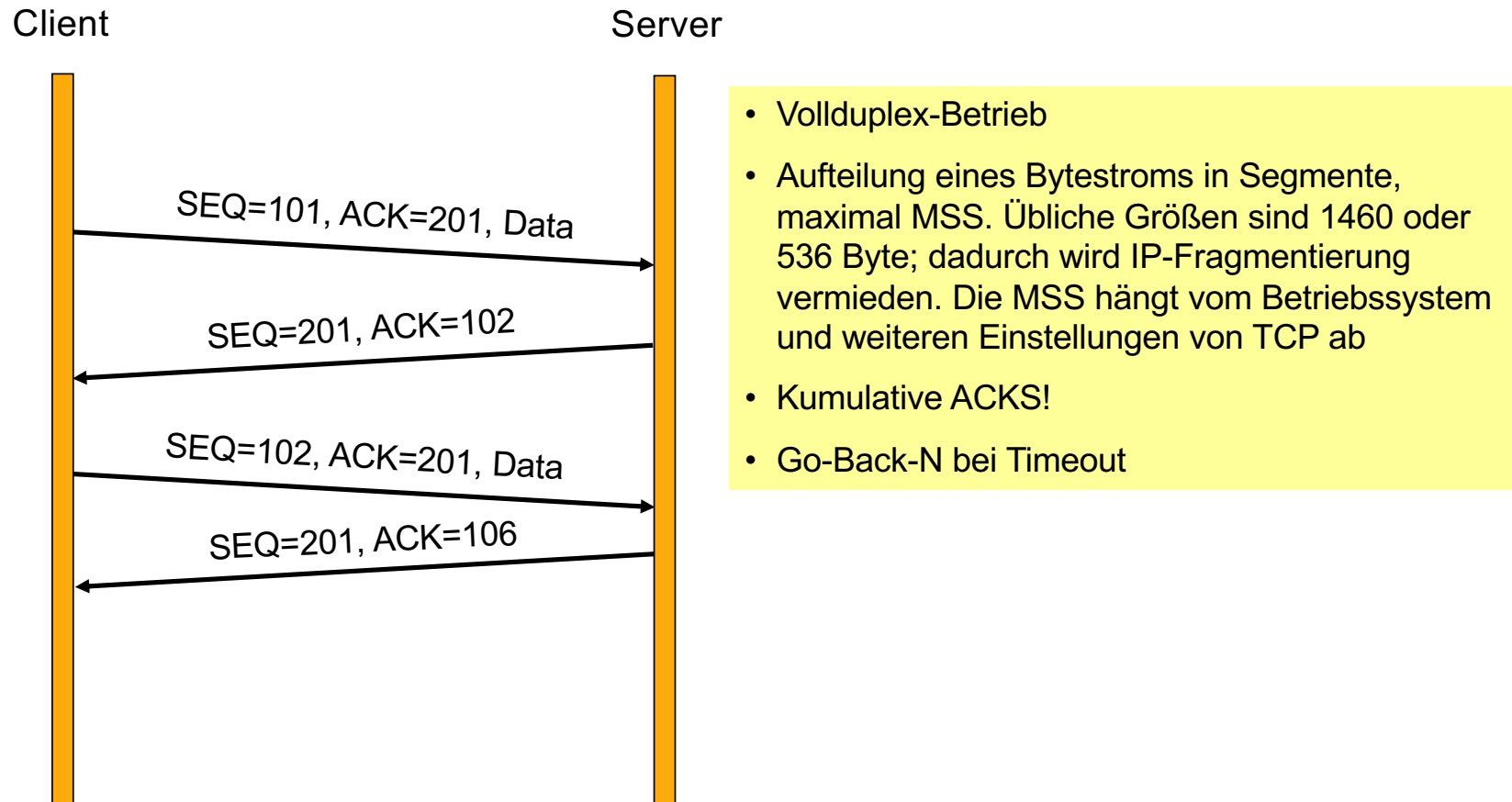
TCP-Verbindungsmanagement: 1. Verbindungsaufbau



- Der Server wartet auf eingehende Verbindungswünsche.
- Der Client führt unter Angabe von IP-Adresse, Portnummer ein **CONNECT** aus
- **CONNECT** sendet ein **SYN** und eine zufällige Sequenznummer mit der gestartet wird (weitere Optionen, z.B. die verwendete MSS)
- Ist der Destination Port der **CONNECT**-Anfrage identisch zu der Port-Nummer, auf der der Server wartet, wird die Verbindung akzeptiert, andernfalls mit **RST** abgelehnt
- Der Server schickt seinerseits das **SYN** zum Client mit einer zufälligen Sequenznummer und bestätigt zugleich den Erhalt des ersten **SYN**-Segments mit einem **ACK** (und seine **MSS**)
- Der Client schickt eine Bestätigung des **SYN**-Segments des Servers. Damit ist die Verbindung aufgebaut und die **MSS** ist das Minimum

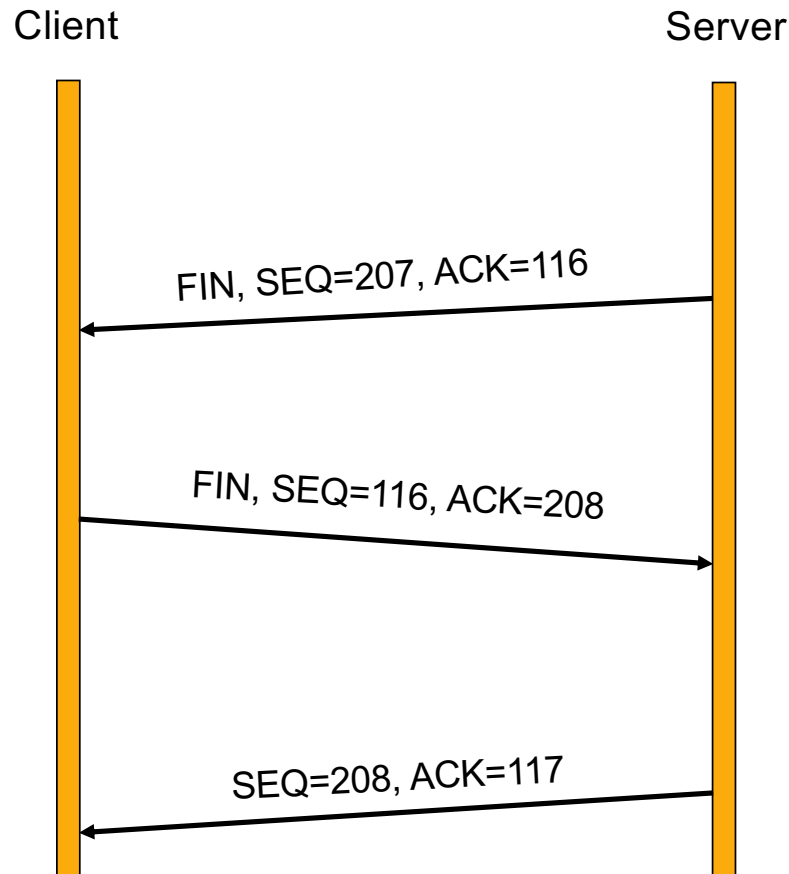


TCP-Verbindungsmanagement: 2. Datenübertragung





TCP-Verbindungsmanagement: 3. Verbindungsende

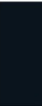


- Abbau als zwei Simplex-Verbindungen
- Senden eines FIN-Segments
- Wird das FIN-Segment bestätigt, wird die Richtung 'abgeschaltet'. Die Gegenrichtung bleibt zunächst noch offen, hier kann eigentlich noch weiter gesendet werden. Spätestens nach einer Karenzzeit (Maximum Segment Lifetime) wird allerdings ein Reset gemacht



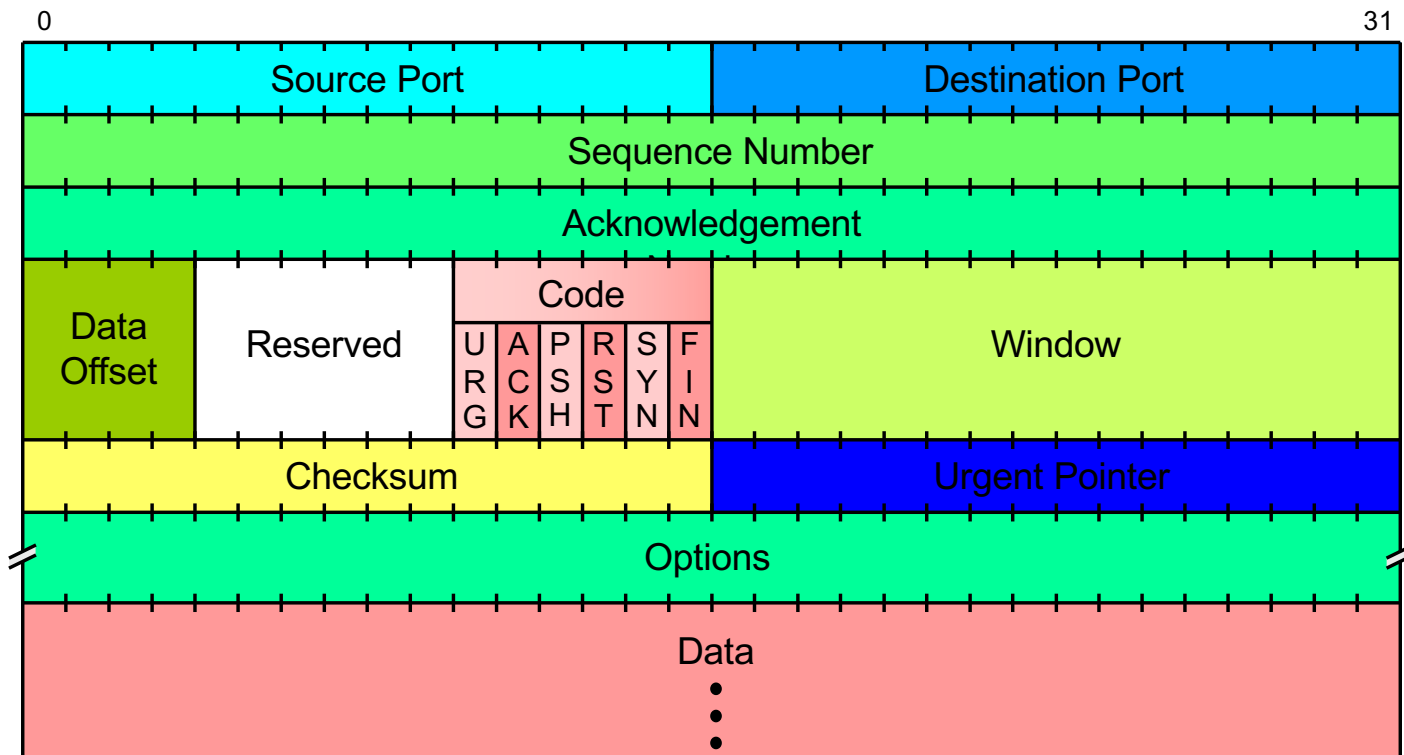
Verbindungsende ist i.Allg. 4-Way Handshake

- Faktisch wird der Verbindungsabbau in beide Richtungen separat signalisiert, also eigentlich ein 4-Way-Handshake
- Warum nicht 3-Way? Beim Verbindungsaufbau (3-Way-Handshake) können SYN und ACK in einem Paket kombiniert werden, weil beide Seiten gleichzeitig die Verbindung aufbauen wollen. Beim Abbau ist das nicht immer möglich: Die Seite, die das erste FIN sendet, signalisiert nur, dass sie keine Daten mehr senden möchte, kann aber weiterhin Daten empfangen. Die andere Seite kann noch Daten senden und erst später ihr eigenes FIN schicken.
- FIN und ACK können zwar kombiniert werden, allerdings sind im Allgemeinen vier Nachrichten notwendig, um sicherzustellen, dass beide Seiten unabhängig voneinander den Verbindungsabbau bestätigen können



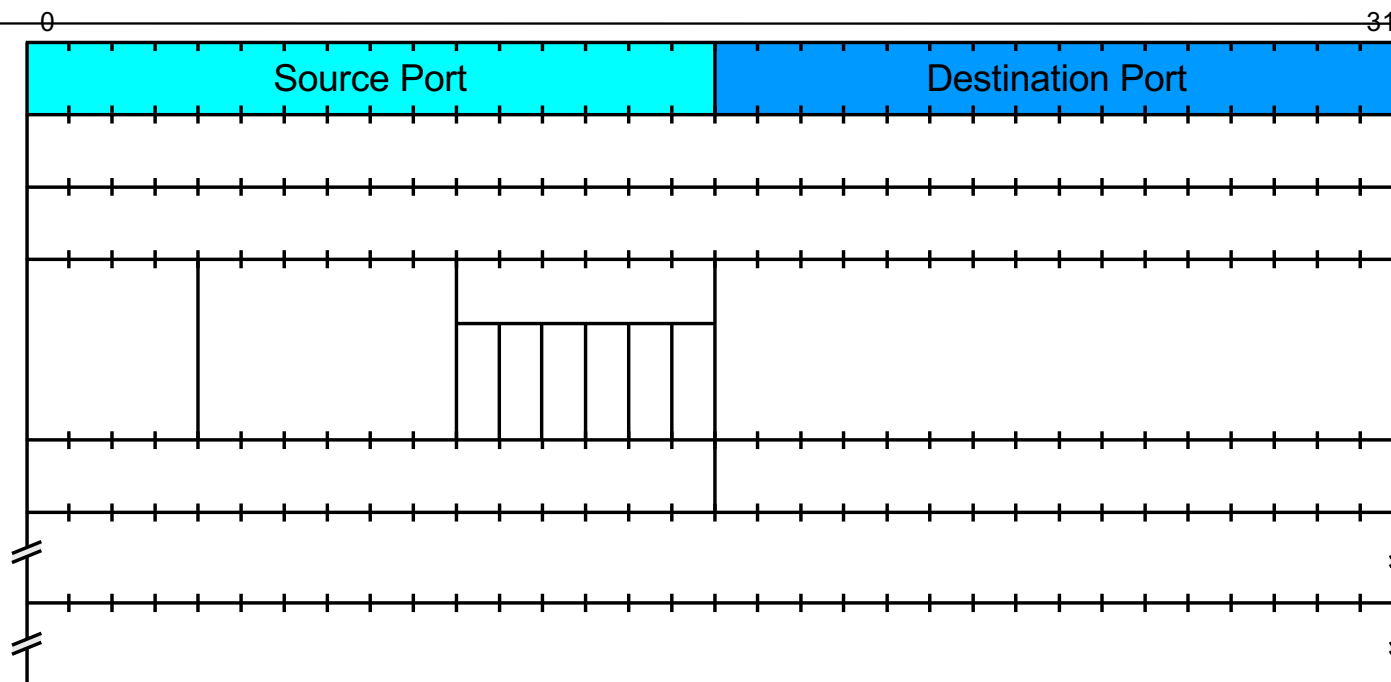


Format eines TCP-Segments





Format eines TCP-Segments



Source Port

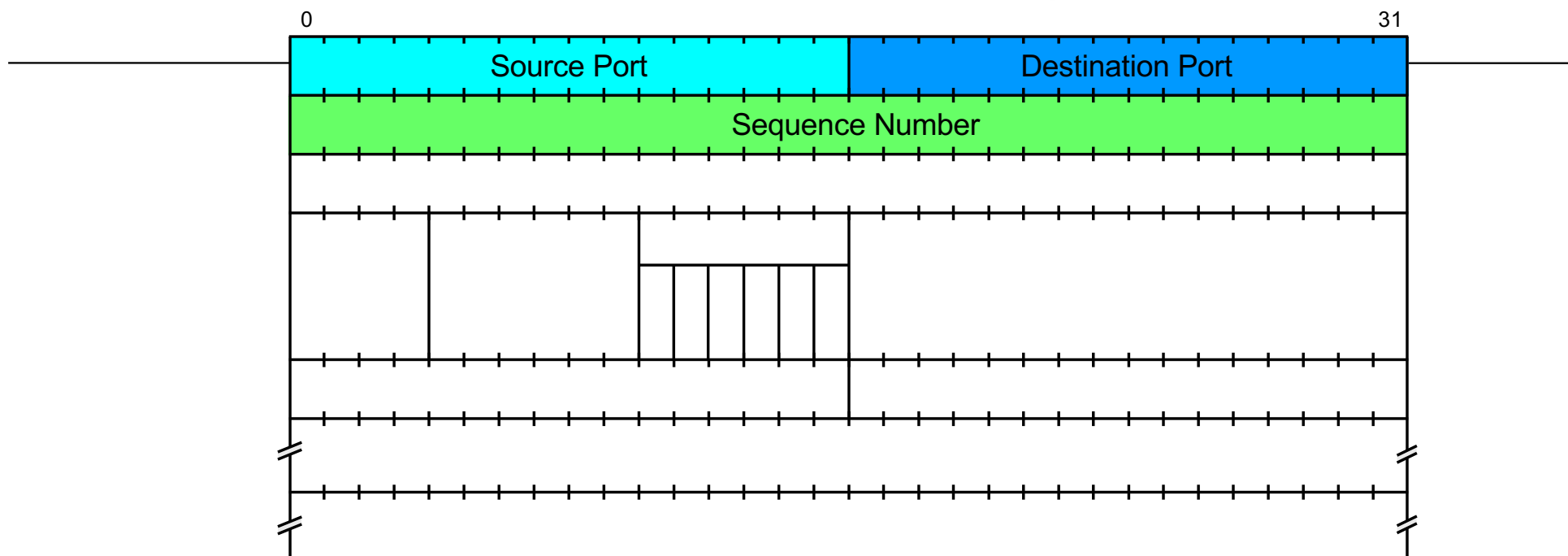
Identifiziert die Anwendung auf der Senderseite.

Destination Port

Identifiziert die Anwendung auf der Empfängerseite.



Format eines TCP-Segments

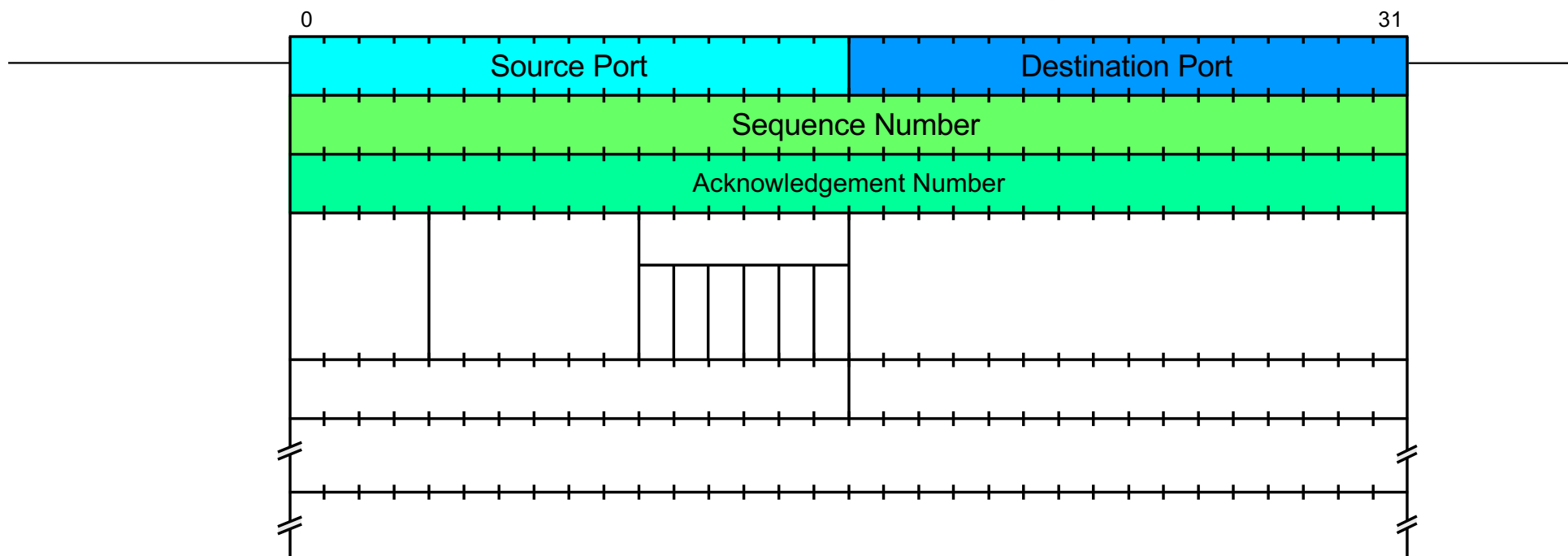


Sequence Number

TCP betrachtet die zu übertragenden Daten als nummerierten Byte-Strom.
Die *Sequence Number* ist die „Position“ des ersten im Segment enthaltenen Datenbytes.



Format eines TCP-Segments

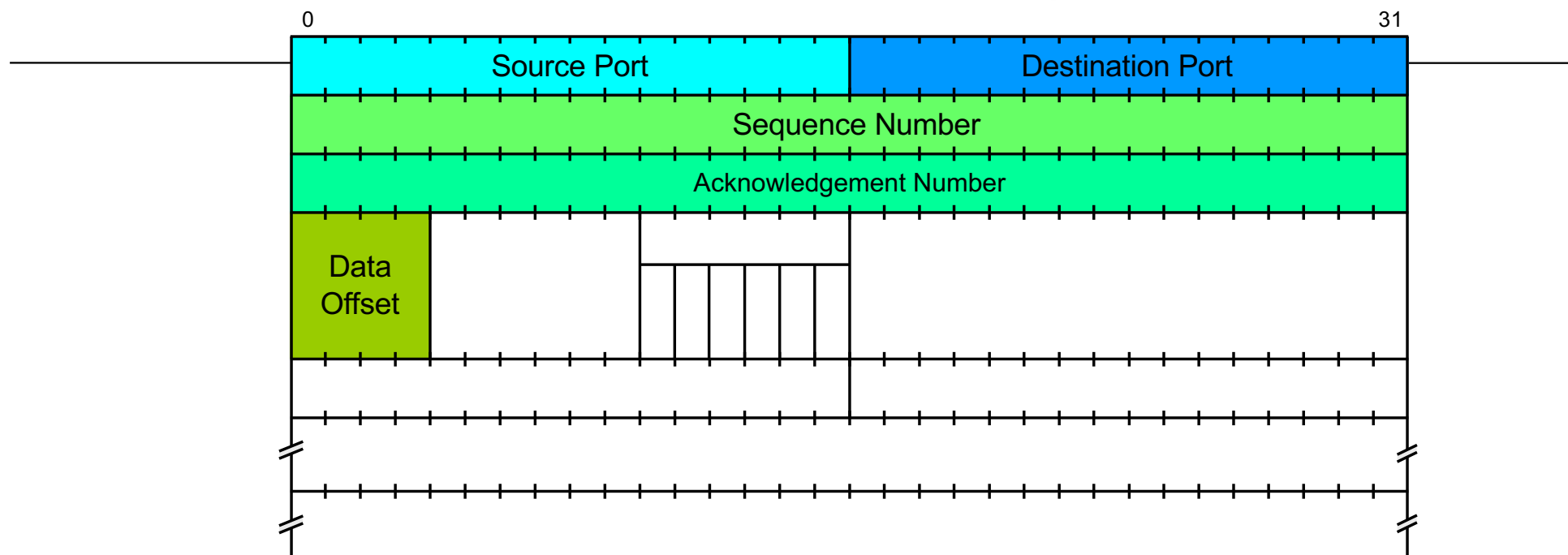


Acknowledgement Number

Dieses Feld bezieht sich auf einen Datenfluss in Gegenrichtung, d.h. hiermit werden Daten bestätigt, die die Station, die das Segment absendet, zuvor von der Zielstation empfangen hat. Es gibt an, welche Position als nächstes erwartet wird



Format eines TCP-Segments

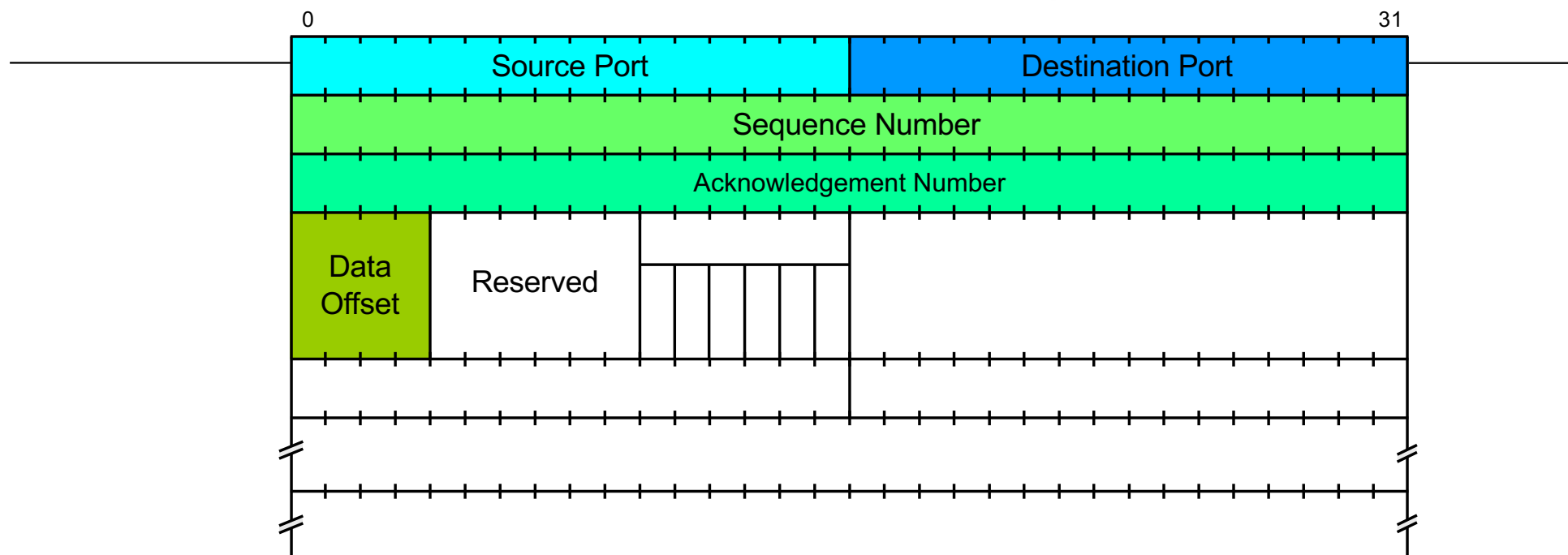


Data Offset

Da der Segment-Header Optionen enthalten kann, ist seine Länge nicht fix. Im *Data Offset*-Feld wird die Länge (d.h. der Beginn des Datenteils) in 32-Bit-Einheiten angegeben.



Format eines TCP-Segments

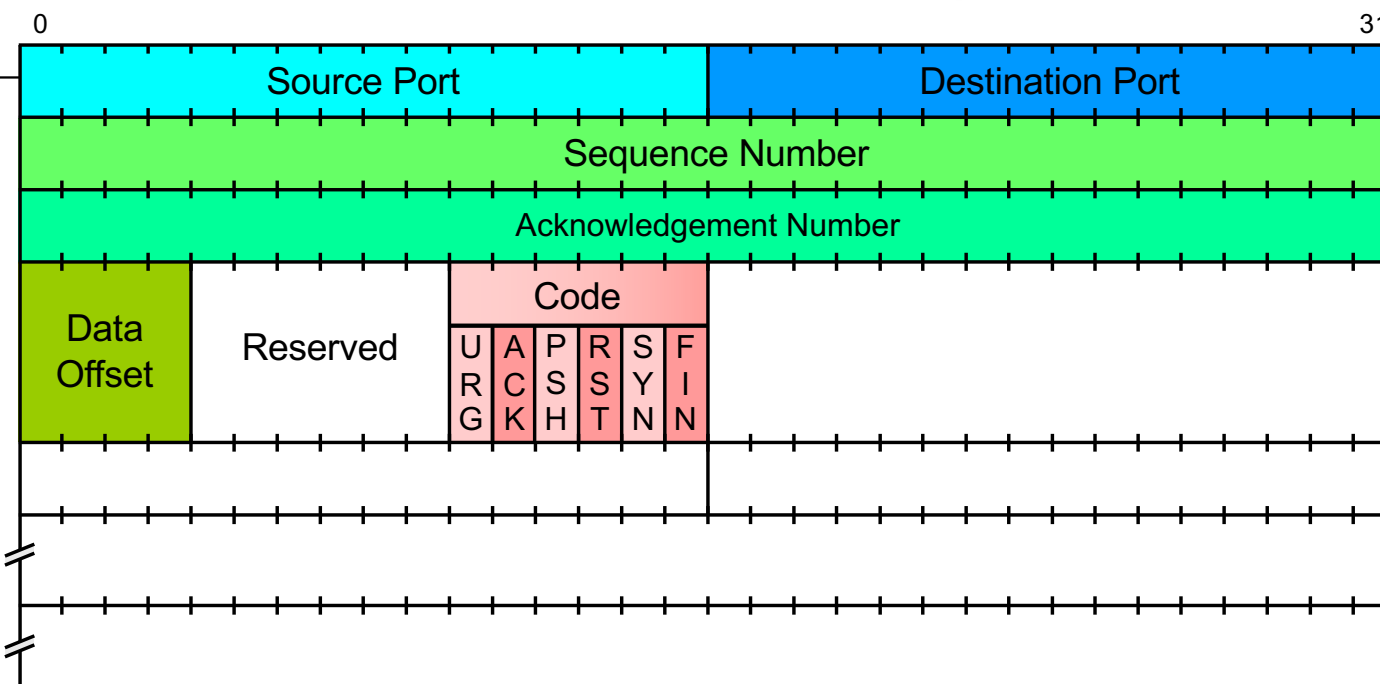


Res.

Reserviert für zukünftige Nutzung.



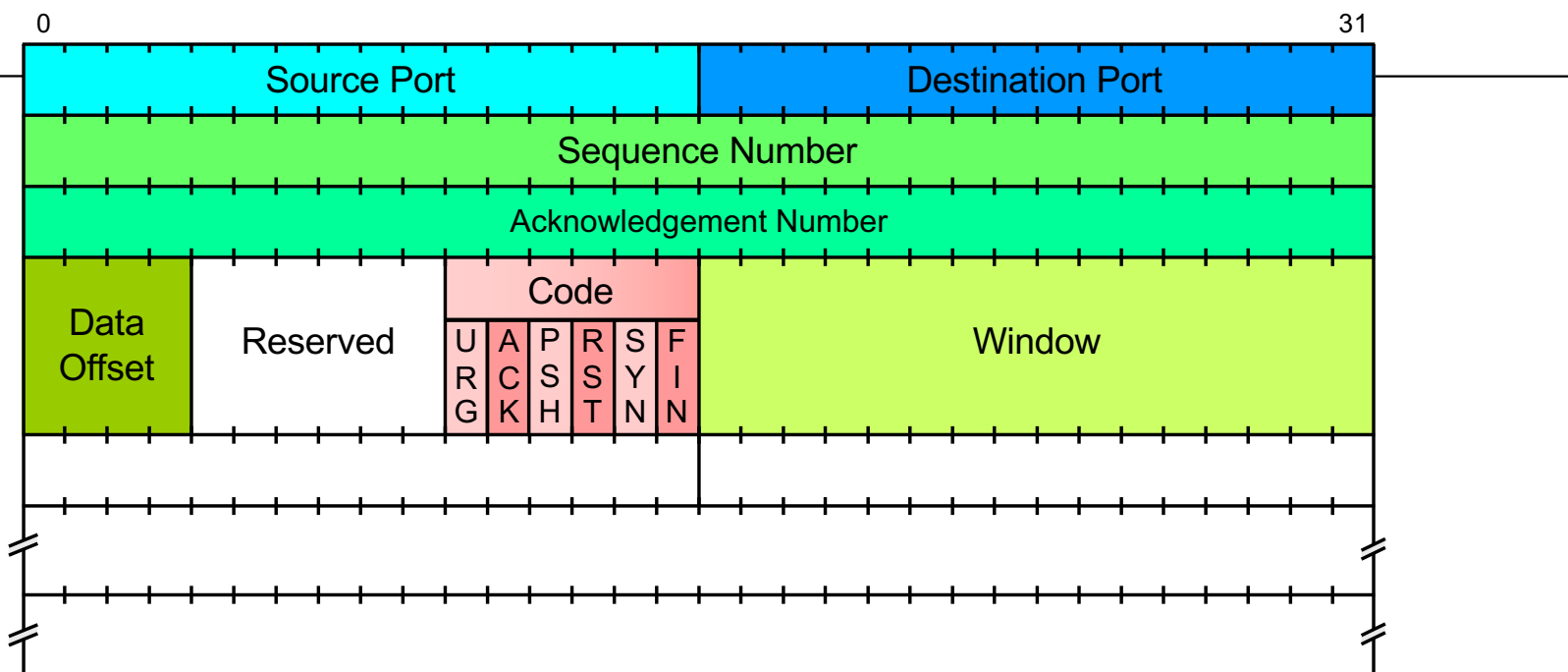
Format eines TCP-Segments



Code	Die Bits des Code-Feldes steuern besondere Funktionen des Segments.			
URG	<i>Urgent Pointer Field is valid</i>	ACK	<i>Acknowledgement Field is valid</i>	
PSH	<i>This segment requests a push</i>	RST	<i>Reset the Connection</i>	
SYN	<i>Synchronize sequence numbers</i>	FIN	<i>Sender has reached end of his byte stream</i>	



Format eines TCP-Segments

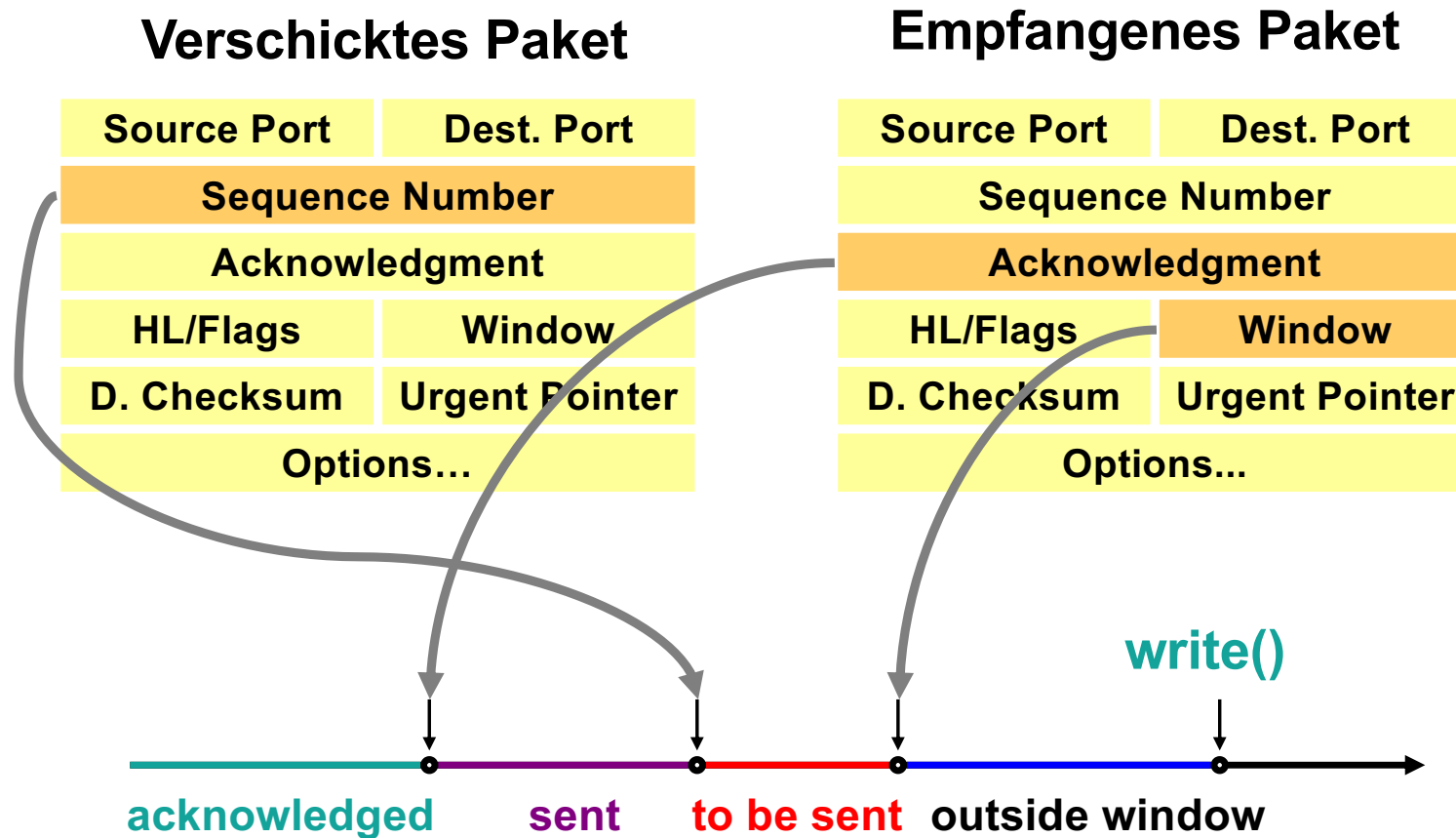


Window

Spezifiziert die Anzahl der Datenbytes (beginnend mit der im *Acknowledgement*-Feld angegebenen Byte-Nummer), die der Sender des Segments als Empfänger eines Datenstromes in Gegenrichtung akzeptieren wird. Anschaulich ist dies der noch freie Platz im Empfangspuffer

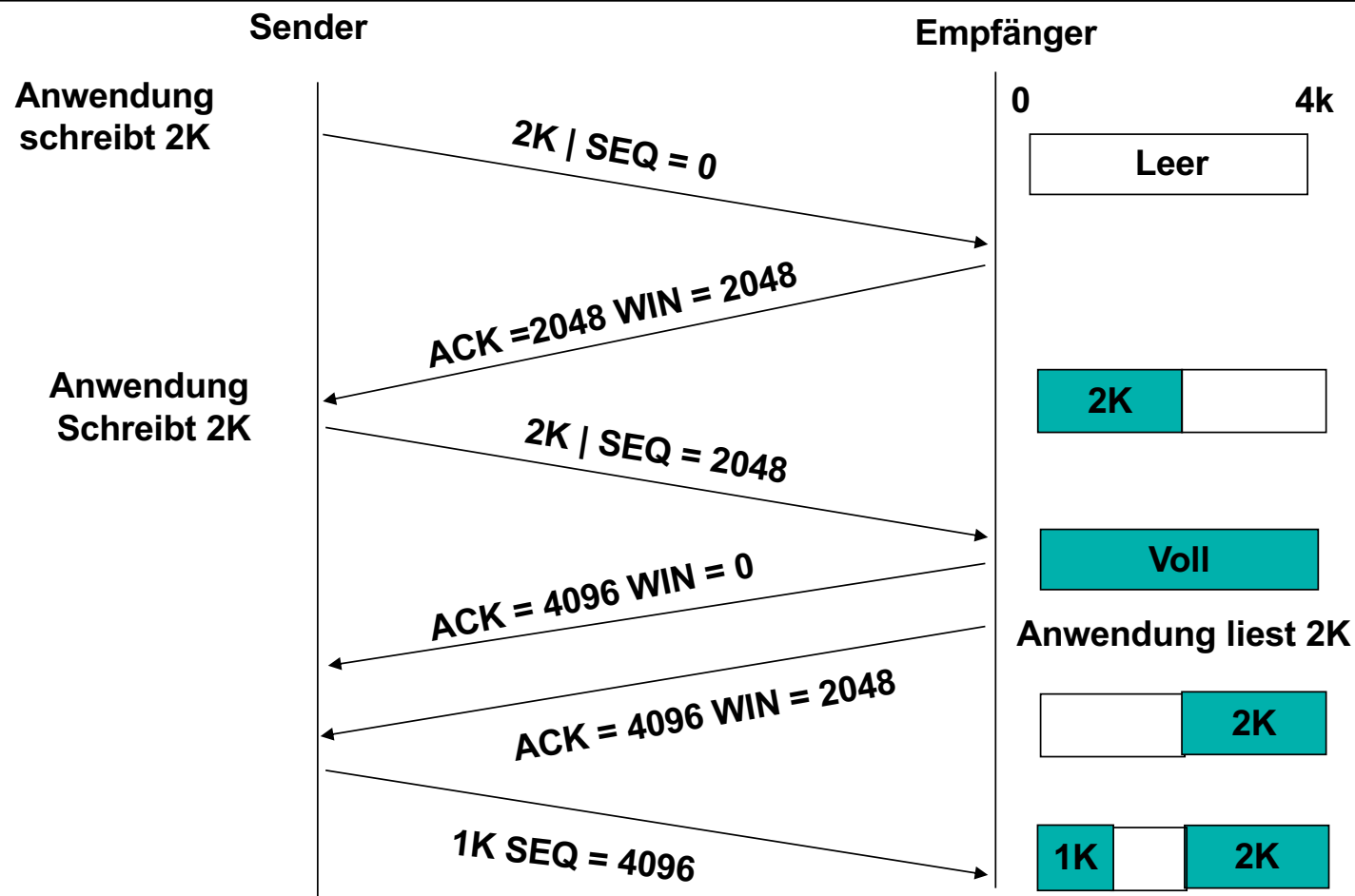


Window-Flußkontrolle: Sender-Seite



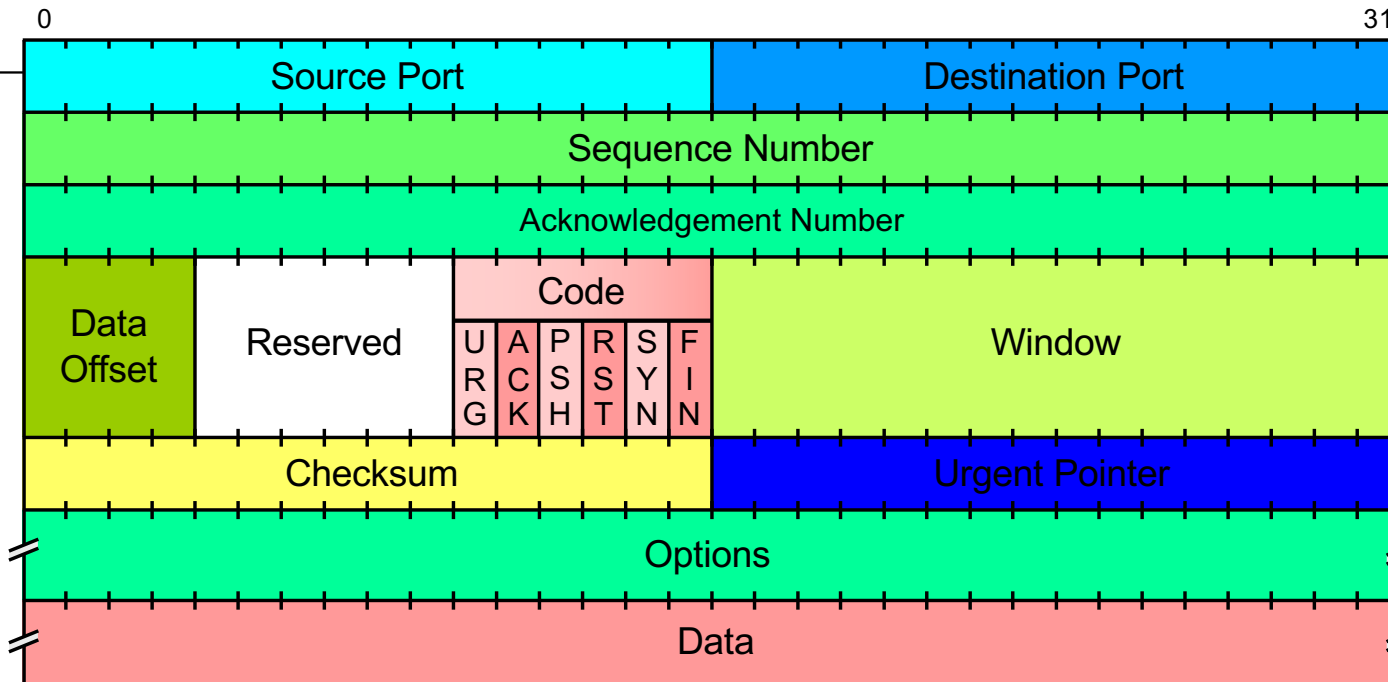


Window-Management in TCP





Format eines TCP-Segments



Checksum

16-Bit Längsparität über das gesamte Segment (*Header* + Daten).

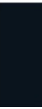
Urgent Pointer

Damit können Teile des zu übertragenden Byte-Stroms als dringend markiert werden.



Push-Flag

- Damit die TCP-Implementierung auf Empfängerseite nicht unnötig lange versucht den Puffer zu füllen (es gibt hier einen Threshold vor Auslieferung) gibt es das PUSH-Flag
- Ein Empfänger, der ein solches Segment empfängt würde dieses auch direkt beim nächsten Read ausliefern, ohne es zu puffern
- Eine Terminal-Emulation wird mit einem CRLF ein Push setzen, damit das Kommando auch beim Empfänger verarbeitet und ausgeführt wird
- Faktisch gibt es keine direkte Möglichkeit dies zu setzen, aber ein Flush auf den Output-Stream ist ein guter Ansatz



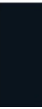


Naggle-Algorithmus

Limitiert die Anzahl der „kleinen“ Segmente

- Solange ein unbestätigtes kleines TCP-Segment unterwegs ist, werden keine weiteren kleinen Segmente gesendet.
- Neue Daten werden im Sendepuffer gesammelt und erst dann verschickt, wenn entweder:
 - eine Bestätigung (ACK) für das vorherige Segment eingetroffen ist,
 - oder genügend Daten angesammelt wurden, um ein volles Segment (meistens MSS, Maximum Segment Size) zu füllen

Kann mit der Socket-Option `TCP_NODELAY` deaktiviert werden. Naggle und Delayed-Ack behindern sich!

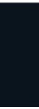




Urgent-Pointer

- Wenn das URG-Flag gesetzt ist interpretiert der Empfänger den Urgent-Pointer und liefert die Daten vor etwaig gepufferten Daten aus
- Die Idee ist, dass beispielsweise interaktive Anwendungen das „überholen“ von Signalen (CTRL-Eingaben) ermöglicht
- Der Urgent-Pointer ist als Offset im Segment gemeint: Ab dieser Stelle beginnen die wichtigen Daten: Diese werden zuerst an die Anwendung ausgeliefert

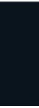
```
socket.sendUrgentData(int data); // 0x03 = Ctrl-C
```





Die Schranke des Bandwidth-Delay-Produkts

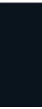
- Der TCP-Header stellt nur ein 16-bit-Feld für das Advertised Window zur Verfügung
- Damit konnten zunächst keine Übertragungsfenster größer 64kB genutzt werden
- Das Bandwidth-Delay-Produkt gibt an, wie groß die Größe des Übertragungsfensters bei gegebener Bandbreite und Round-Trip-Zeit sein muss





Wie kann das Übertragungsfenster vergrößert werden?

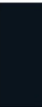
- Einführen einer **Window-Scale-Erweiterung** die ein Skalierungsfaktor für das 16-Bit Window-Feld darstellt
 - Der **Skalierungsfaktor** wird als neue TCP-Option eingeführt: Window Scale ist 3 Byte lang (zzgl. eines Füll-Byte)
 - Das 3. Byte gibt die Skalierung LOGARITHMISCH an
 - Anschaulich werden so viele binäre Nullen das Window-Header-Feld gesetzt
 - Wertebereich ist 0x00-0x0E (Wert 14!)
- Die Option wird nur in einem SYN-Segment **beim Verbindungsaufbau übertragen**. Danach wird der Wert für die ganze Verbindung angenommen
- Beide Seiten müssen eine Window-Scale-Option verschicken Null bedeutet "keine Skalierung"
- Die neue maximale Fenstergröße in Bytes errechnet sich wie folgt:
$$65536 * 2^{14} = 65535 * 16384 = 1.073.725.440$$
- TCP-intern wird die Fenstergröße jetzt als 30-Bit-Zahl verwaltet





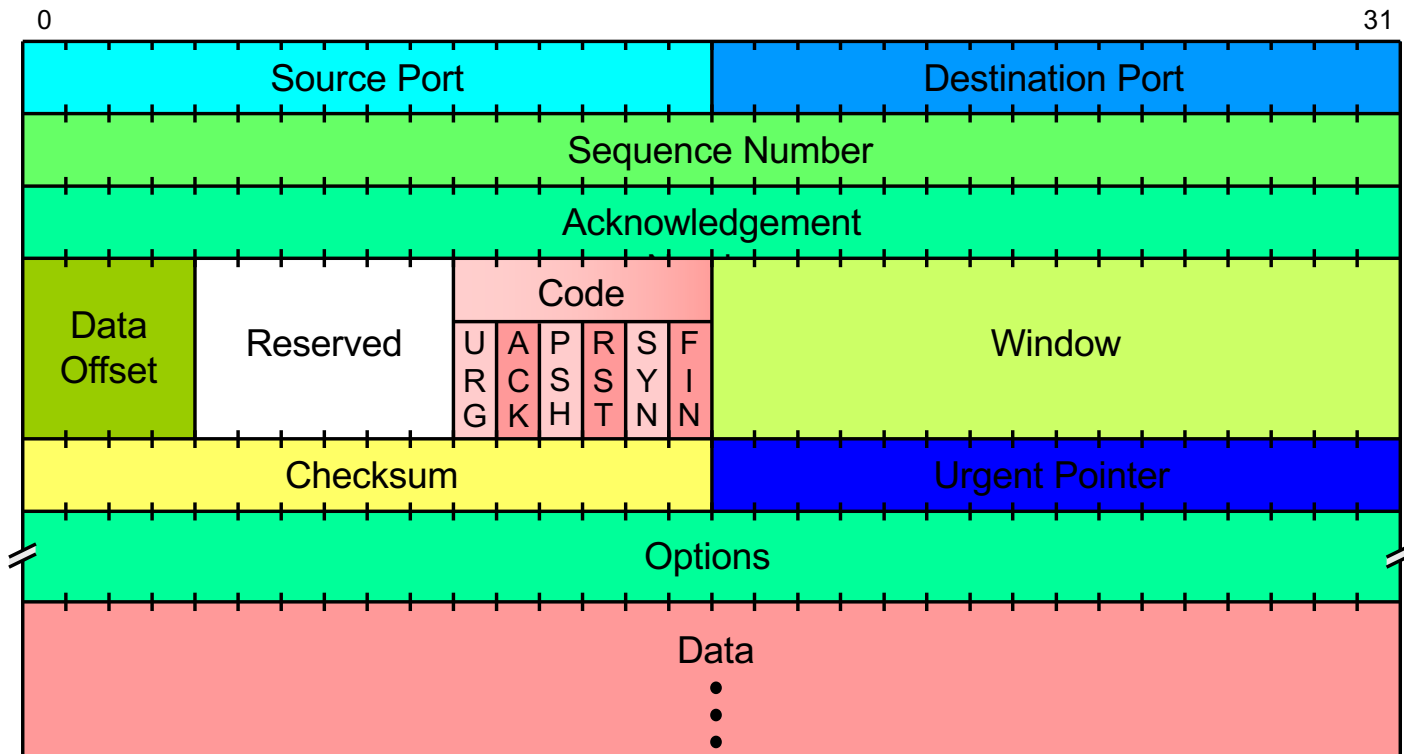
Merke

- Ohne Window-Scale-Option ist TCP nicht geeignet für sogenannte Long-Fat-Pipes
- Die Window-Scale Option wird beim Aufbau der Verbindung **in beiden Richtungen** separat mitgeteilt
- Anschaulich handelt es sich um die Anzahl der binären Nullen, die hinter dem Advertised/Receiver Window zusätzlich interpretiert werden
- Damit können aber auch nur noch größere Änderungen im Puffer des Empfängers angezeigt werden und nicht mehr das Lesen einzelner Bytes
- Verpasst eine Anwendung das Setzen der Option vor einer Verbindung, so kann sie ggf. die gewünschte Rate nicht erzielen





Format eines TCP-Segments



Frage: Wie wird die Länge eines Datagramms ermittelt?



Was kommt am Rechner an?

